

FAR Chunking & Vector Database Guide

FAR Chunking & Vector Database Guide

A step-by-step guide for the Acquisitions team on how the FAR Bot processes and searches the Federal Acquisition Regulation.

Part 1: Development Implementation (Free & Open-Source)

Why Free & Open-Source? This prototype was built using 100% free, open-source tools to enable rapid development, easy experimentation, and zero licensing costs. All libraries can run locally without cloud dependencies.

Technology Stack (All Free & Open-Source)

Tool	Purpose	Why Chosen	License
BeautifulSoup	HTML parsing	Industry standard, easy to use	MIT
lxml	Fast HTML/XML parser	Faster than Python's default parser	BSD
Sentence-Transformers	Text embeddings	Pre-trained models, no GPU required	Apache 2.0
FAISS	Vector similarity search	Facebook's battle-tested library	MIT
OpenAI API	Response generation	Pay-per-use, no license fees	N/A

Total cost to build: \$0 (excluding OpenAI API usage for chat responses)

Overview: How It Works

Pipeline Flow:

FAR HTML Files (11,685 files) → **Parse & Chunk** (split_text.py) →
Create Vectors (vectorize_text.py) → **Search & Answer** (far_chatbot.py)

Result: 3,893 searchable FAR section chunks stored in a FAISS vector database

Step 1: Parsing HTML Files

File: python/split_text.py

The FAR is published as individual HTML files (e.g., 52.212-4.html). The first step extracts structured text from these HTML files.

What Happens

```
from bs4 import BeautifulSoup

# Read HTML file
soup = BeautifulSoup(html_content, "lxml")

# Extract main heading (e.g., "52.212-4 Contract Terms and
# Conditions")
heading = soup.find('h1', class_='title')

# Extract subsections (e.g., (a), (b), (c)...)
subsections = soup.find_all('p', class_='ListL1')
```

Output Format

Each HTML file produces a chunk text file:

Heading: 52.212-4 Contract Terms and Conditions
Text: (a) Inspection/Acceptance. The Contractor shall only
tender...
 (b) Assignment. The Contractor or its assignee...
 (c) Change...

Key Design Decisions

Decision	Why
Use lxml parser	Faster and more reliable than default HTML parser
Extract headings separately	Preserves document structure for better search

Decision	Why
Keep subsections together	Maintains semantic context of related content

Step 2: Creating Vector Embeddings

File: python/vectorize_text.py

This step converts text into numerical vectors that AI can search by meaning.

What Happens

1. Load the embedding model

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
```

2. Combine heading + text

```
combined_text = f"{heading} {section_text}"
```

3. Generate embeddings

```
embeddings = model.encode(texts) # → 384-dimensional vectors
```

4. Save to FAISS index

```
import faiss
index = faiss.IndexFlatL2(384) # L2 (Euclidean) distance
index.add(embeddings)
faiss.write_index(index, "faiss_index.index")
```

Output Files

File	Description
dita_html/faiss_index.index	Binary file with 3,893 vectors
dita_html/texts.txt	Plain text of each chunk (one per line)

Why These Choices?

Choice	Reason
Sentence-BERT (paraphrase-MiniLM-L6-v2)	Optimized for semantic similarity, fast, small
384 dimensions	Good balance of accuracy vs. speed
FAISS IndexFlatL2	Simple, accurate, no training needed
L2 distance	Works well for normalized embeddings

Step 3: Searching the Vector Database

File: python/far_chatbot.py

When a user asks a question, the system finds the most relevant FAR sections.

Search Process

```
def search_similar(self, query: str, top_k: int = 50):  
    # 1. Expand query with synonyms  
    expanded_query = self.expand_query(query)  
    # "sole source" → "sole source noncompetitive single  
    #    source..."  
  
    # 2. Convert query to vector  
    query_embedding = self.model.encode([expanded_query])  
  
    # 3. Search FAISS for similar vectors  
    distances, indices = self.faiss_index.search(query_embedding,  
        top_k)  
  
    # 4. Convert distances to similarity scores  
    results = []  
    for distance, idx in zip(distances[0], indices[0]):  
        similarity = 1 / (1 + distance) # Higher = more similar  
        results.append((self.texts[idx], similarity))  
  
    return results
```

Query Enhancement Features

Feature	How It Works
Query Expansion	Adds synonyms: “SAT” → “simplified acquisition threshold”
Context Boosting	Recent topics get higher scores in follow-ups
Cross-References	Finds related sections mentioned in results

How to Run the Pipeline (Development)

1. Install Dependencies

```
pip install sentence-transformers faiss-cpu beautifulsoup4 lxml  
openai python-dotenv
```

2. Parse HTML Files

```
python python/split_text.py  
# Output: dita_html/chunks/*.txt
```

3. Create Vector Index

```
python python/vectorize_text.py  
# Output: dita_html/faiss_index.index, dita_html/texts.txt
```

4. Run the Chatbot

```
python python/far_chatbot.py  
# Interactive: Ask questions about FAR!
```

Part 1 Summary

Metric	Value
Source HTML files	11,685
Searchable chunks	3,893
Vector dimensions	384
Embedding model	paraphrase-MiniLM-L6-v2 (free)
Vector index	FAISS IndexFlatL2 (free)
Search time	~10ms per query
Total licensing cost	\$0

Part 2: Production Implementation (AWS)

Scaling to Production: When moving from prototype to enterprise deployment, we upgrade to managed AWS services, better embedding models, and production-grade infrastructure.

Production Architecture

AWS Architecture Flow:

S3 Bucket (FAR HTML) → AWS Lambda / ECS (Processing) → Amazon OpenSearch (Vectors)

Upgraded Components

1. Better Embedding Models

Model	Dimensions	Quality	Speed	Best For
paraphrase-MiniLM-L6-v2 (current)	384	☆☆☆	Fast	Prototyping
all-mpnet-base-v2	768	☆☆☆☆	Medium	Balanced
amazon.titan-embed-text-v2	1024	☆☆☆☆☆	Medium	Production
text-embedding-3-large (OpenAI)	3072	☆☆☆☆☆	Fast	Highest quality

Recommended for Production: Amazon Titan Embeddings v2 - Native AWS integration - FedRAMP authorized - 1024 dimensions for better semantic understanding

```
import boto3
import json

bedrock = boto3.client('bedrock-runtime', region_name='us-east-1')

def get_embedding(text):
    response = bedrock.invoke_model(
        modelId='amazon.titan-embed-text-v2:0',
        body=json.dumps({
            "inputText": text,
            "dimensions": 1024, # Can reduce to 512/256 for speed
            "normalize": True
        })
    )
    return json.loads(response['body'].read())['embedding']
```

2. Amazon OpenSearch Service (Vector Database)

Why OpenSearch over FAISS for Production?

Feature	FAISS (Dev)	OpenSearch (Prod)
Scalability	Single machine	Distributed cluster

Feature	FAISS (Dev)	OpenSearch (Prod)
Updates	Rebuild entire index	Real-time inserts
Access control	None	IAM, fine-grained
Backup/Recovery	Manual	Automated snapshots
Monitoring	None	CloudWatch integration
Hybrid search	Vector only	Vector + keyword

Setting Up OpenSearch with k-NN

```

from opensearchpy import OpenSearch, RequestsHttpConnection
from requests_aws4auth import AWS4Auth
import boto3

# AWS authentication
credentials = boto3.Session().get_credentials()
awsauth = AWS4Auth(
    credentials.access_key,
    credentials.secret_key,
    'us-east-1',
    'es',
    session_token=credentials.token
)

# Connect to OpenSearch
client = OpenSearch(
    hosts=[{'host': 'your-domain.us-east-1.es.amazonaws.com',
            'port': 443}],
    http_auth=awsauth,
    use_ssl=True,
    verify_certs=True,
    connection_class=RequestsHttpConnection
)

# Create index with k-NN enabled
index_body = {
    "settings": {
        "index": {
            "knn": True,
            "knn.algo_param.ef_search": 100
        }
    },
    "mappings": {
        "properties": {
            "embedding": {
                "type": "knn_vector",
                "dimension": 1024,
                "method": {
                    "name": "hnsw",
                    "space_type": "cosinesimil",
                    "engine": "nmslib",

```

```

        "parameters": {
            "ef_construction": 128,
            "m": 24
        }
    },
    "text": {"type": "text"},
    "section_number": {"type": "keyword"},
    "part": {"type": "keyword"},
    "title": {"type": "text"},
    "last_updated": {"type": "date"}
}
}
}

```

```
client.indices.create(index='far-vectors', body=index_body)
```

Indexing FAR Chunks

```

def index_far_chunk(section_number, title, text, embedding):
    document = {
        "section_number": section_number,
        "title": title,
        "text": text,
        "embedding": embedding,
        "part": section_number.split('.')[0],
        "last_updated": datetime.now().isoformat()
    }

    client.index(
        index='far-vectors',
        id=section_number,
        body=document,
        refresh=True
    )

```

Semantic Search Query

```

def search_far(query_text, top_k=20):
    # Get embedding for query
    query_embedding = get_embedding(query_text)

    # Hybrid search: k-NN + keyword boost
    search_body = {
        "size": top_k,
        "query": {
            "bool": {
                "should": [
                    # Vector similarity (primary)
                    {
                        "knn": {
                            "embedding": {

```



```

        "vector": query_embedding,
        "k": top_k
    }
},
# Keyword match (boost exact terms)
{
    "multi_match": {
        "query": query_text,
        "fields": ["text^2", "title^3",
"section_number^4"],
        "boost": 0.3
    }
}
]
},
"_source": ["section_number", "title", "text"]
}

response = client.search(index='far-vectors',
    body=search_body)
return response['hits']['hits']

```

3. LLM Response Generation (Amazon Bedrock)

Replace OpenAI with AWS-native LLM for FedRAMP compliance:

```

def generate_far_response(query, context_chunks):
    # Build context from search results
    context = "\n\n".join([
        f"[{chunk['section_number']}] {chunk['text']}"
        for chunk in context_chunks
    ])

    prompt = f"""You are an expert Federal Acquisition Regulation
        consultant.
        Answer the following question using ONLY the FAR sections
        provided.
        Always cite specific FAR section numbers.

        FAR Context:
        {context}

        Question: {query}

        Answer: """

    response = bedrock.invoke_model(
        modelId='anthropic.claude-3-sonnet-20240229-v1:0',
        body=json.dumps({
            "anthropic_version": "bedrock-2023-05-31",

```

```
        "max_tokens": 2000,
        "messages": [
            {"role": "user", "content": prompt}
        ]
    })
)

result = json.loads(response['body'].read())
return result['content'][0]['text']
```

Production Cost Estimate

Service	Usage Estimate	Monthly Cost
OpenSearch (r6g.large.search, 2 nodes)	~4K vectors	~\$200-300
Amazon Bedrock - Titan Embeddings	10K queries/month	~\$10-20
Amazon Bedrock - Claude 3 Sonnet	10K queries/month	~\$30-50
Lambda + API Gateway	10K requests/month	~\$5-10
S3 Storage	1 GB FAR content	~\$0.02
Total		~\$250-400/month

Costs can be reduced with reserved instances and right-sizing.

Feature Comparison: Dev vs. Production

Feature	Dev (Part 1)	Production (Part 2)
Embedding Model	MiniLM-L6 (384d)	Titan v2 (1024d)
Vector Storage	FAISS (local file)	OpenSearch (managed)
LLM	OpenAI GPT-4	Claude 3 on Bedrock
Authentication	None	IAM + API Keys
Compliance	N/A	FedRAMP ready
Scalability	Single user	Enterprise
Updates	Rebuild index	Real-time
Monitoring	Logs only	CloudWatch + X-Ray
Hybrid Search	Vector only	Vector + Keyword
Cost	~\$0 (API usage)	~\$300/month

Advanced Production Features

1. Chunking Improvements

```
# Use semantic chunking instead of section-based
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n\n", "\n", ". ", " "]
)
```

2. Re-ranking for Better Results

```
# Use Cohere or cross-encoder for re-ranking top results
from sentence_transformers import CrossEncoder

reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
scores = reranker.predict([(query, chunk) for chunk in
                           top_chunks])
```

3. Metadata Filtering

```
# Filter by FAR part before vector search
search_body = {
    "query": {
        "bool": {
            "must": [{"knn": {...}}],
            "filter": [
                {"term": {"part": "52"}}, # Only Part 52 clauses
                {"range": {"last_updated": {"gte": "2024-01-01"}}}
            ]
        }
    }
}
```

4. Caching Frequent Queries

```
# Use ElastiCache for repeated queries
import redis

cache = redis.Redis(host='your-elasticache.amazonaws.com')

def cached_search(query):
    cache_key = f"far_search:{hash(query)}"
    cached = cache.get(cache_key)
    if cached:
        return json.loads(cached)
```

```
results = search_far(query)
cache.setex(cache_key, 3600, json.dumps(results)) # 1 hour
TTL
return results
```

Migration Path: Dev → Production

1. **Phase 1:** Deploy OpenSearch cluster, migrate vectors
 2. **Phase 2:** Switch to Titan embeddings, re-embed all chunks
 3. **Phase 3:** Replace OpenAI with Bedrock Claude
 4. **Phase 4:** Add API Gateway + authentication
 5. **Phase 5:** Implement monitoring and alerting
-

Summary

	Part 1 (Dev)	Part 2 (Production)
Goal	Prove the concept works	Enterprise deployment
Cost	Free (+ OpenAI API)	~\$300/month
Time to build	1-2 days	2-4 weeks
Best for	Demos, testing, learning	Agency-wide deployment

Questions?

- **Dev Source Code:** /Users/collinschreyer/GSA/FAR_BOT/python/
- **Dev Vector Index:** /Users/collinschreyer/GSA/FAR_BOT/dita_html/faiss_index.index
- **FAR HTML Files:** /Users/collinschreyer/GSA/FAR_BOT/dita_html/